

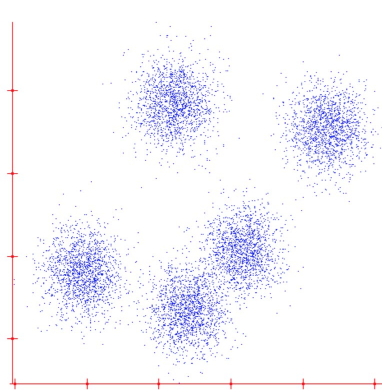
K-Means Clustering

From intuition to objective functions and model selection

March 27, 2026

Why We Cluster

- ▶ Clustering is **unsupervised**: we observe feature vectors, but no labels.
- ▶ The goal is to create groups with high within-cluster similarity and low between-cluster similarity.
- ▶ Common use cases include customer segmentation, document organization, image grouping, social network analysis, and genomics.

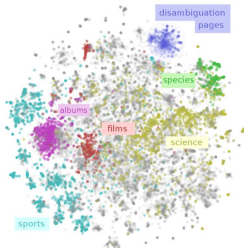


No labels $\implies$ discover structure from geometry alone

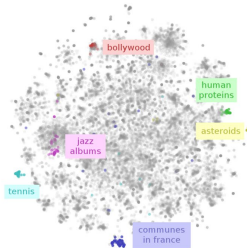
Clustering Shows Up in Many Domains

Documents and networks

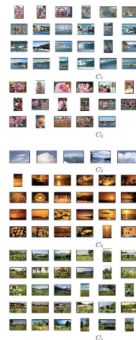
Large Clusters



Small Clusters



Images and visual categories



[Goldberger et al.]

- ▶ The objects being clustered can be customers, articles, images, genes, or users in a network.
- ▶ K-means is one way to turn those feature vectors into coherent groups.

Representative Applications of Clustering

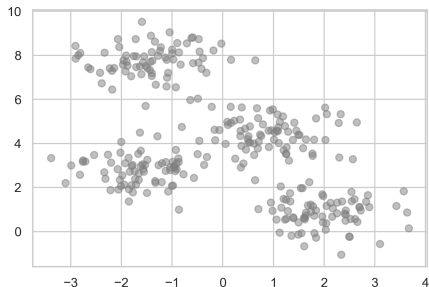
- ▶ **Customer segmentation:** group customers by behavior, preferences, or demographics.
- ▶ **Document clustering:** organize large text corpora into coherent topics.
- ▶ **Anomaly detection:** identify unusual behavior that may indicate fraud or cyberattacks.
- ▶ **Social network analysis:** find communities with similar links or interests.
- ▶ **Genomics:** discover genes with similar expression patterns.
- ▶ **Urban planning:** group regions with similar land use, demographics, or crime patterns.

Why this matters

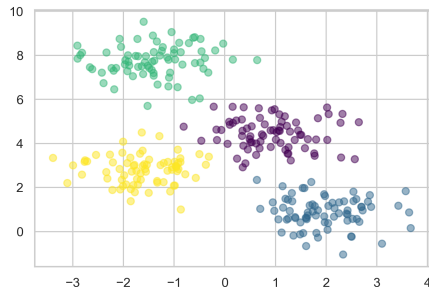
Clustering is useful whenever we want structure without labels: find groups, summarize populations, or flag unusual points.

What K-Means Produces

Before



After



- ▶ The algorithm assigns every point to exactly one cluster and summarizes each cluster by its center.
- ▶ The only thing it sees is distance, so the choice of features and scaling matters.

K-means objective

$$W(\mathcal{C}, \mu_1, \dots, \mu_k) = \sum_{\ell=1}^k \sum_{i: C(i)=\ell} \|x_i - \mu_\ell\|^2$$

- ▶ $\mathcal{C}$ is the cluster assignment map and μ_ℓ is the centroid of cluster ℓ .
- ▶ Minimizing squared Euclidean distances makes clusters **compact** around their centers.
- ▶ For a fixed assignment, the best centroid is the sample mean:

$$\mu_\ell = \frac{1}{n_\ell} \sum_{j: C(j)=\ell} x_j, \quad n_\ell = \#\{i : C(i) = \ell\}.$$

- ▶ This quantity is also called the **within-cluster scatter** or **WCSS**.

Why the Mean Appears

$$\mu_\ell^\star = \arg \min_{m \in \mathbb{R}^d} \sum_{i: C(i)=\ell} \|x_i - m\|^2 = \frac{1}{n_\ell} \sum_{i: C(i)=\ell} x_i$$

Interpretation

With squared distance, the center that minimizes total within-cluster error is the arithmetic mean. That is why the algorithm is called **k-means**.

- ▶ Assignment step: keep the centroids fixed and send each point to its nearest center.
- ▶ Update step: keep the assignments fixed and recompute each center as the mean of its assigned points.
- ▶ Each step decreases the objective, so the algorithm moves downhill until it reaches a local optimum.

Why We Need an Iterative Heuristic

- ▶ Searching over *all* possible clusterings is combinatorial.
- ▶ The number of ways to partition n points into k nonempty clusters is the Stirling number:

$$S(n, k) = \frac{1}{k!} \sum_{\ell=1}^k (-1)^{k-\ell} \binom{k}{\ell} \ell^n$$

- ▶ For example, $S(10, 4) = 34105$ and $S(19, 4) \approx 10^{10}$.
- ▶ Exhaustive optimization is unrealistic, so we use a fast greedy procedure instead.

Lloyd's algorithm

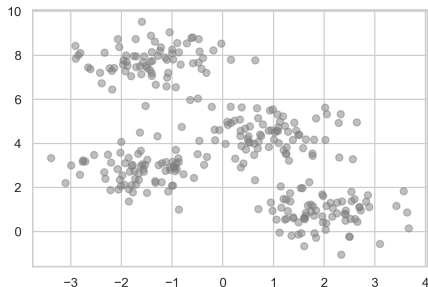
$$C(i) \leftarrow \arg \min_{\ell} \|x_i - \mu_{\ell}\|^2$$

$$\mu_{\ell} \leftarrow \frac{1}{n_{\ell}} \sum_{j: C(j)=\ell} x_j$$

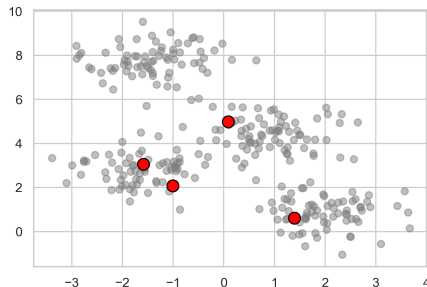
- ▶ Initialize k centroids.
- ▶ Alternate assignment and update.
- ▶ Stop when assignments no longer change.

A Concrete Run of Lloyd's Algorithm

1. Choose $k = 4$

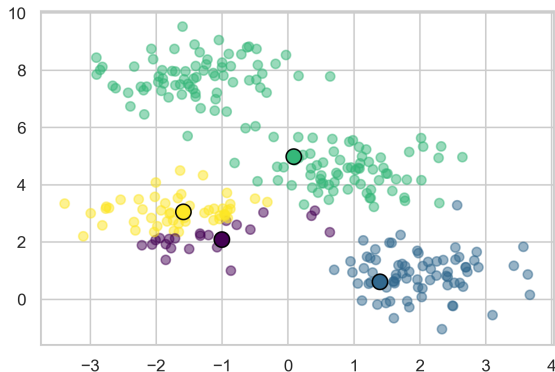


2. Randomly initialize centroids



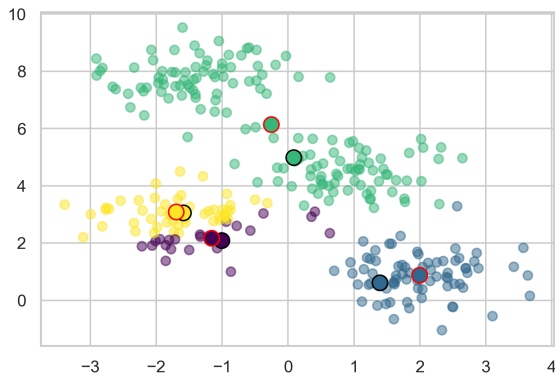
- ▶ We start with a chosen value of k and an initial guess for the k centers.
- ▶ From here, the algorithm alternates between assignment and update.

Iteration 1: Assign Every Point to Its Closest Center



- ▶ Each point is assigned to the nearest current centroid.
- ▶ This creates a provisional partition of the data.

Iteration 1: Recompute the Centroids



- ▶ Each centroid moves to the mean of the points currently assigned to it.
- ▶ In the plot, the red outlined markers show the newly computed centers.

The Reassignment-Update Loop

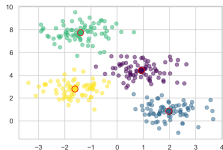
Another assignment step



The following update



Near convergence

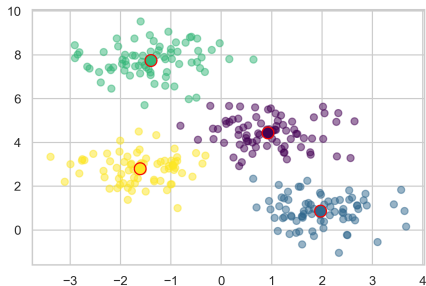


The final update

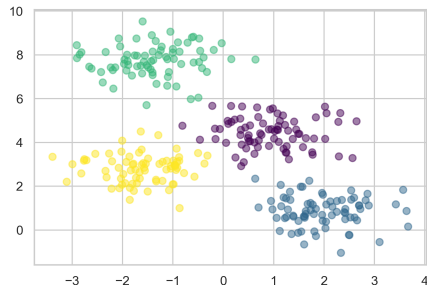


- ▶ Early on, assignments and centers can move a lot.
- ▶ As the algorithm settles, each update becomes smaller.

Stop when the centers no longer move

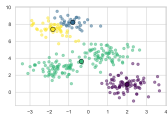
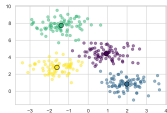


Final cluster assignment



- ▶ Once the updated centers match the current centers, the algorithm has converged.
- ▶ The result is a locally optimal partition for this initialization.

Different random starts can lead to different answers



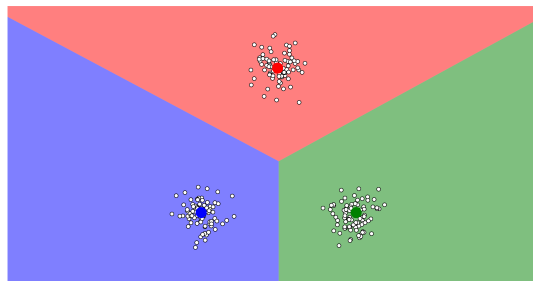
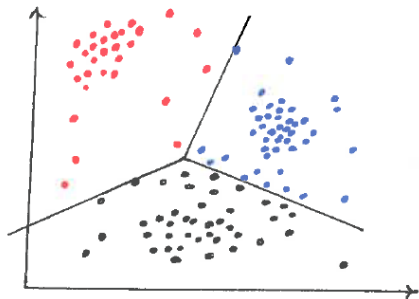
- ▶ K-means converges to a **local** optimum, so different starts can produce different partitions.
- ▶ A common fix is to run the algorithm many times and keep the best objective value.
- ▶ Intuition: if one region already has a center, we want the next center to land in a region that is still far away.

k-means++ idea

Pick the first center uniformly. For each candidate point x , define $D(x) = \min_{j \in \{1, \dots, i\}} \|x - m_j\|$, where $m_1, \dots, m_i$ are the i centers already chosen.

So $D(x)$ is the distance from x to its nearest chosen center, and the next center is sampled with $\Pr(x \text{ is chosen next}) \propto D^2(x)$. Points far from the current centers are therefore more likely to start a new cluster.

Cluster Geometry



- ▶ With fixed centroids, k-means partitions the space into **Voronoi cells**: every point goes to its nearest center.
- ▶ Boundaries are piecewise linear, so clusters tend to be convex and roughly spherical in Euclidean space.
- ▶ That geometry makes elongated, imbalanced, and non-convex shapes harder to recover.

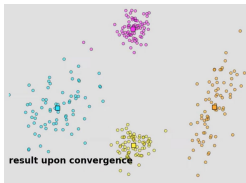
When K-Means Works and When It Fails

Works well when

- ▶ clusters are compact and reasonably separated
- ▶ Euclidean distance is meaningful
- ▶ features are standardized and on comparable scales
- ▶ you want a fast baseline or preprocessing step

Struggles when

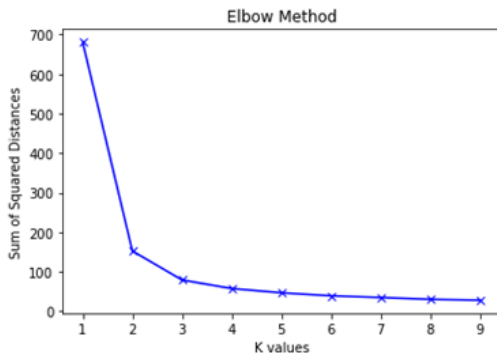
- ▶ clusters are non-convex or have very different sizes
- ▶ there are strong outliers
- ▶ the true structure is density-based rather than centroid-based
- ▶ the value of k is poorly chosen



Choosing k with WCSS and the Elbow Method

$$\text{WCSS} = \sum_{j=1}^K \sum_{x_i \in C_j} \|x_i - \mu_j\|^2$$

- ▶ Smaller WCSS means tighter clusters.
- ▶ WCSS always decreases as K increases.
- ▶ The elbow is where extra clusters give only a small additional improvement.
- ▶ The bend is a reasonable place to stop adding clusters.



Choosing k with Silhouette Scores

$$a(i) = \frac{1}{|C|-1} \sum_{x_j \in C, j \neq i} d(x_i, x_j)$$

$$b(i) = \min_{C' \neq C} \frac{1}{|C'|} \sum_{x_j \in C'} d(x_i, x_j)$$

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}, \quad S = \frac{1}{n} \sum_{i=1}^n s(i)$$

- ▶ $a(i)$ measures cohesion inside the assigned cluster.
- ▶ $b(i)$ measures separation from the nearest competing cluster.
- ▶ $s(i) \approx 1$: the point is well matched to its own cluster.
- ▶ $s(i) \approx 0$: the point lies near a boundary between clusters.
- ▶ $s(i) < 0$: the point may fit better in a different cluster.

Larger average silhouette means the clustering has both tighter groups and better separation between them.

Using WCSS and Silhouette Together

WCSS

- ▶ Measures **compactness** inside each cluster.
- ▶ Great for elbow plots and fast screening of candidate k values.
- ▶ Limitation: it ignores separation between different clusters.

Silhouette

- ▶ Measures **compactness plus separation**.
- ▶ Great for comparing cluster quality more directly.
- ▶ Limitation: it is more computationally expensive.

- ▶ In practice, start with WCSS to narrow the range of plausible k values.
- ▶ Then use silhouette plots, domain knowledge, and visual inspection to decide whether the clusters are actually meaningful.

Interpreting the Clusters

- ▶ K-means gives **partitions**, not semantic labels. Interpretation still requires subject-matter judgment.
- ▶ Inspecting cluster centroids often reveals the dominant profile of each group.
- ▶ In high dimensions, visualization often uses PCA, t-SNE, or UMAP after clustering.
- ▶ A good workflow is to standardize features, compare several k values, inspect centroids and example members, then validate with domain knowledge.



Takeaways

- ▶ K-means is simple and fast because it alternates between assignment and least-squares centroid updates.
- ▶ Its objective is to minimize within-cluster squared distances to the centroids.
- ▶ Its Euclidean geometry favors compact, convex groups.
- ▶ Initialization and model selection matter, which is why k-means++ and validation tools are so important.
- ▶ Use it as a strong baseline, then move beyond it when the data violate its assumptions.